PageRank com MPI e Apache Spark: comparação de desempenho

Trabalho 2 de Sistemas Distribuídos

Universidade Federal Fluminense

Caio Ferreira · Leonardo Peçanha · Lucas Soares

Outubro de 2026

Resumo. Implementamos o PageRank em três versões (sequencial, MPI com mpi4py e Apache Spark com PySpark) e medimos tempo, speedup e eficiência de 1 a 128 processos num nó de 16 núcleos físicos, sobre um grafo sintético de 1.000.000 páginas e 9.993.543 links. Também medimos três alterações de código, nas mesmas três áreas para as duas abordagens. A melhor versão MPI levou 0,326 s para 10 iterações, contra 22,1 s da melhor versão Spark, uma diferença de 68 vezes. O Spark mostrou speedup relativo maior, mas a partir de um tempo absoluto muito mais alto. As alterações que mais ajudaram foram as que reduziram a comunicação no MPI e a troca de dados entre executores no Spark.

Código-fonte: https://github.com/LeoHazer/pagerank-mpi-spark

1. Introdução

O enunciado pede um programa em MPI e outro em Spark, com pelo menos duas primitivas coletivas e vetores grandes trocados entre os processos, além de um relatório com speedup e eficiência para 1, 2, 4, ... 128 processos e com os tempos de três alterações de código que tiveram impacto no desempenho. Seguimos a forma de comparação de Saxena et al. [1], que compararam MPI e Spark em aplicações de ciência de dados escritas em Python.

Escolhemos o PageRank [2] porque ele é iterativo e, a cada iteração, todos os processos precisam do vetor de notas inteiro, com uma posição por página. Isso leva naturalmente ao uso de coletivas e à troca de um vetor grande: com um milhão de páginas, são 8 MB trocados por iteração.

2. O problema: PageRank

O PageRank modela uma pessoa que navega ao acaso: em cada página ela segue um dos links com probabilidade d = 0,85 ou abre uma página qualquer com probabilidade 1 - d. O PageRank de uma página é a chance de encontrar essa pessoa nela depois de muito tempo. O cálculo é feito por iterações: todas as páginas começam com nota $1/N$ e, a cada rodada, cada página divide sua nota igualmente entre as páginas para as quais aponta:

$$PR_{t+1}(v) = \frac{1-d}{N} + d\left(\sum_{u \to v} \frac{PR_t(u)}{g(u)} + \frac{M(t)}{N}\right)$$

em que $g(u)$ é o número de links que saem de u e $M(t)$ é a soma das notas das páginas sem links de saída, repartida entre todas para que a soma das notas continue igual a 1. As iterações param quando a soma das diferenças entre duas rodadas fica abaixo de 10^{-10}.

Usamos um grafo sintético de 1.000.000 páginas, gerado com semente fixa para ser reprodutível. A origem de cada link é sorteada de forma uniforme e o destino é sorteado com viés para os números baixos, o que cria poucas páginas muito populares, como na web. Depois de retirar links repetidos e de uma página para ela mesma, restaram 9.993.543 links; 46 páginas não têm links de saída. O grafo é guardado como uma lista de arestas (dois vetores de inteiros de 64 bits, origem e destino), formato fácil de dividir entre processos. Com tolerância de 10^{-10}, o PageRank desse grafo converge em 21 iterações.

3. Implementações

3.1 Versão sequencial

A versão sequencial usa NumPy. A cada iteração, calcula a parte da nota que viaja por cada link (nota da origem dividida pelo grau da origem) e soma essas partes por destino com `np.bincount`. Ela é a referência de correção: os testes comparam o resultado com o da biblioteca networkx, e todas as outras versões são comparadas com ela.

3.2 Versão MPI

A versão MPI usa mpi4py com buffers NumPy, que são enviados sem serialização. O rank 0 lê o grafo e reparte os links em pedaços de tamanho igual com `Scatterv`. A cada iteração, cada processo calcula um vetor parcial de tamanho N com o que os seus links entregam a cada página, e um `Allreduce` soma esses vetores. Como todos recebem o total, cada processo aplica a fórmula sozinho e todos decidem juntos quando parar, sem comunicação extra. A Tabela 1 resume as coletivas usadas.

Coletiva	Quem envia / recebe	Dados	Quando
bcast	rank 0 para todos	número de páginas e de links	uma vez
Scatterv	rank 0 reparte	origens e destinos dos links (80 MB cada no total)	uma vez
Bcast	rank 0 para todos	grau de saída de cada página (8 MB)	uma vez
Allreduce	todos somam e todos recebem	vetor de contribuições de tamanho N (8 MB)	a cada iteração
reduce	todos para o rank 0	tempo do processo mais lento	no fim
Barrier	todos	sincronização antes de medir	antes de cada medição

Tabela 1. Primitivas coletivas da versão MPI.

3.3 Versão Spark

A versão Spark usa a API de DataFrames do PySpark, em que as operações rodam dentro do Spark e não passam linha a linha pelo Python. O grafo vira três tabelas guardadas em memória (`cache`): links com origem, destino e peso (1/grau da origem), páginas, e páginas sem saída. Cada iteração faz um *join* dos links com a tabela de notas, soma por destino com *groupBy* e faz um *left join* com as páginas, para incluir as que não recebem links. Como o Spark adia os cálculos e guarda o histórico de todas as operações, cada iteração termina com `localCheckpoint`, que calcula a tabela na hora e descarta esse histórico. O número de partições é igual ao número de núcleos, seguindo [1], que igualou as partições ao número de executores.

3.4 Validação

O projeto tem 60 testes automáticos. Todas as versões, incluindo as alteradas, precisam produzir o mesmo vetor da versão sequencial (diferença L1 menor que 10^{-9}), com vários números de processos e em casos-limite, como processos que ficam sem nenhum link. Um desses testes encontrou um erro real: um processo sem links enviava inteiros no `Allreduce` enquanto os outros enviavam reais, e o programa travava.

4. Ambiente e metodologia

As medições foram feitas no nó n-11 do cluster da disciplina: dois processadores Intel Xeon E5-2680 de 8 núcleos cada (16 núcleos físicos, 32 lógicos com *hyper-threading*) e 31 GB de memória, com Python 3.12.3, NumPy 2.5.4, mpi4py 4.1.2 sobre Open MPI 4.1.6, PySpark 4.2.0 e Java 21. O outro nó (n-10) estava fora do ar durante o trabalho, então todas as medições usaram um nó só: o MPI se comunica pela memória do próprio nó e o Spark roda em modo local (`local[N]`), com N núcleos.

Todas as execuções fazem 10 iterações fixas. Como cada iteração faz o mesmo trabalho, o tempo por iteração é constante e o speedup não depende de quantas iterações rodamos; a correção do resultado convergido é garantida pelos testes. Medimos só o **tempo de cálculo** (as iterações), sem a leitura do arquivo e a distribuição do grafo, de forma parecida com [1], que excluiu o tempo de inicialização. No MPI, os processos se sincronizam antes de medir e vale o tempo do mais lento.

Cada caso do MPI foi repetido 3 vezes e cada caso do Spark 2 vezes, e usamos a média; a maior variação entre rodadas foi de 4,8% da média. O MPI foi de 1 a 128 processos; acima de 32, os processos dividem núcleos. No Spark paramos em 32 núcleos, e medimos 64 uma vez para registrar que não havia ganho, como o enunciado permite. O speedup de cada versão é $S(p) = T(1) / T(p)$, em relação a ela mesma com 1 processo, e a eficiência é $E(p) = S(p) / p$. O nó estava sem carga no início, e as medições levaram 49 minutos.

5. Resultados das versões originais

A Tabela 2 mostra tempo, speedup e eficiência das versões originais. A versão sequencial levou 1,849 s.

Processos	MPI: tempo	MPI: speedup	MPI: eficiência	Spark: tempo	Spark: speedup	Spark: eficiência
1	1,782 s	1,00	100,0%	85,8 s	1,00	100,0%
2	1,187 s	1,50	75,0%	56,3 s	1,53	76,3%
4	0,697 s	2,56	63,9%	39,6 s	2,17	54,2%
8	0,851 s	2,09	26,2%	28,4 s	3,02	37,7%
16	1,406 s	1,27	7,9%	24,7 s	3,47	21,7%
32	2,635 s	0,68	2,1%	23,2 s	3,70	11,5%
64	5,072 s	0,35	0,5%	24,3 s	3,52	5,5%
128	9,684 s	0,18	0,1%	-	-	-

Tabela 2. Versões originais, grafo de 1.000.000 páginas, 10 iterações (média das rodadas; "-" = não medido).

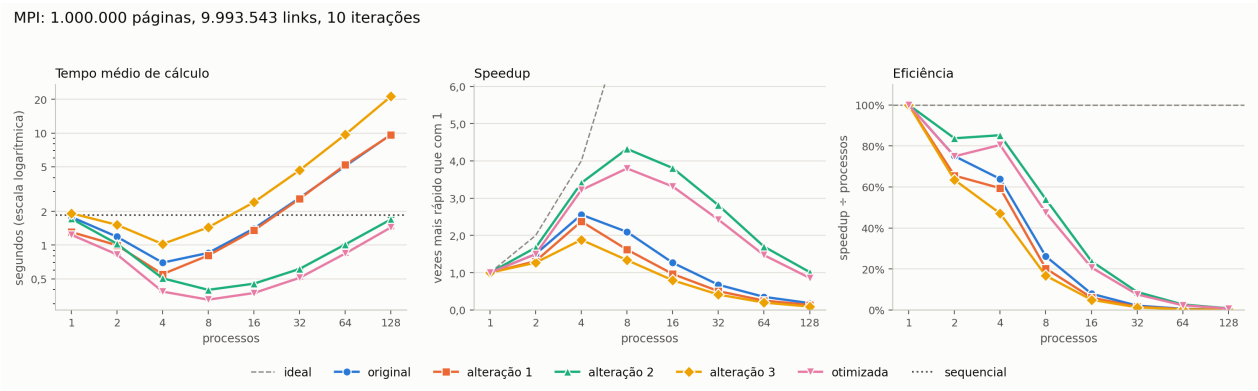


Figura 1. MPI: tempo, speedup e eficiência da versão original e das alterações. A linha pontilhada no primeiro painel é a versão sequencial.

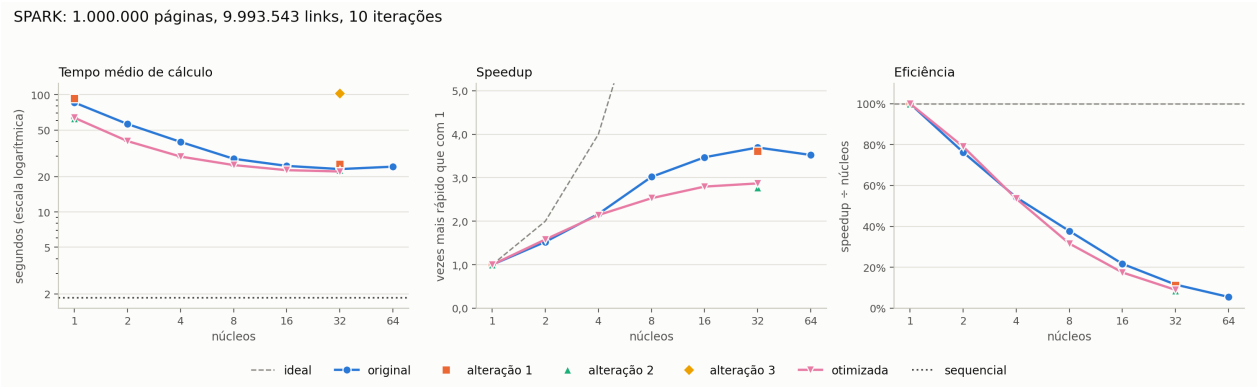


Figura 2. Spark: o mesmo para o Spark. As alterações 1, 2 e 3 foram medidas só em alguns números de núcleos e aparecem como pontos isolados.

MPI. O MPI original fica mais rápido até 4 processos (0,697 s, speedup 2,56) e depois volta a ficar mais lento, chegando a 9,684 s com 128. Há três motivos. Primeiro, o `Allreduce` soma um vetor de 8 MB de todos os processos a cada iteração, e esse custo cresce com o número de processos enquanto a parte de cálculo de cada um diminui. Segundo, todos os processos repetem as operações sobre o vetor inteiro (aplicar a fórmula e medir a variação), uma parte que não se divide. Terceiro, os processos dividem a mesma memória: em testes preliminares num notebook, cópias independentes do mesmo cálculo, sem

MPI, ficaram proporcionalmente mais lentas quando rodaram juntas, porque o PageRank passa a maior parte do tempo lendo e escrevendo posições espalhadas da memória. Acima de 16 processos entra ainda o *hyper-threading* e, acima de 32, processos dividindo núcleos.

Spark. O Spark original vai de 85,8 s com 1 núcleo para 23,2 s com 32 (speedup 3,70), e com 64 núcleos não melhora (24,3 s). Com 32 núcleos, cada iteração leva cerca de 2,3 s, um piso que vem da coordenação: a cada iteração o Spark planeja e executa vários *jobs*, reorganiza os dados entre as partições e espera todas as tarefas terminarem.

Comparação. O Spark tem speedup relativo maior que o do MPI original, mas parte de um tempo muito mais alto. A melhor versão Spark (22,1 s) é 68 vezes mais lenta que a melhor versão MPI (0,326 s), e até a versão sequencial é 12 vezes mais rápida que o Spark com 32 núcleos. Como em [1], o MPI foi muito mais rápido. Diferente de [1], o MPI não escalou melhor que o Spark: com um grafo deste tamanho, cada iteração do MPI leva menos de 0,2 s mesmo com 1 processo, e a comunicação passa a pesar logo com poucos processos.

6. Alterações de desempenho

Fizemos três alterações nas mesmas três áreas para as duas abordagens, cada uma medida sozinha contra a versão original, e uma versão "otimizada" com as alterações 1 e 2 juntas. A alteração 3 é uma piora proposital, para medir o custo da serialização, que é o tema de uma das análises de [1]. Os arquivos originais não foram modificados; as alterações ficam em arquivos separados (`pagerank_alterado.py`).

	Área	MPI	Spark
1	Trabalho repetido em cada iteração	o peso de cada link (1/grau da origem) é calculado uma vez, antes das iterações	a soma das páginas sem saída é anotada durante o cálculo principal (observe), eliminando um job por iteração
2	Divisão dos dados	cada processo fica dono de uma faixa de páginas de destino e recebe só os links que chegam nelas; o Allreduce de N posições vira um Allgatherv	links divididos por destino e tabela de notas enviada inteira a cada executor (broadcast), sem reorganizar os links a cada iteração
3	Formato dos dados	allreduce minúsculo: o vetor é serializado (pickle) antes de ser enviado	RDD de tuplas do Python no lugar de DataFrame
1 + 2	Versão otimizada	alterações 1 e 2 juntas	alterações 1 e 2 juntas

Tabela 3. As três alterações em cada abordagem.

Versão	1 proc.	4 proc.	8 proc.	32 proc.	128 proc.
original	1,782 s	0,697 s	0,851 s	2,635 s	9,684 s
alteração 1	1,307 s	0,550 s	0,807 s	2,587 s	9,630 s
alteração 2	1,723 s	0,505 s	0,399 s	0,613 s	1,696 s
alteração 3	1,924 s	1,022 s	1,438 s	4,655 s	21,2 s
otimizada	1,237 s	0,384 s	0,326 s	0,510 s	1,438 s

Tabela 4. Tempo de cálculo das versões MPI.

Versão	1 núcleo	32 núcleos
original	85,8 s	23,2 s
alteração 1	92,8 s	25,7 s
alteração 2	63,0 s	22,7 s
alteração 3	-	102,6 s
otimizada	63,6 s	22,1 s

Tabela 5. Tempo de cálculo das versões Spark ("-" = não medido).

6.1 Alteração 1: trabalho repetido em cada iteração

No MPI, calcular o peso de cada link uma vez deixou a versão 27% mais rápida com 1 processo e 21% com 4. Com muitos processos o ganho some, porque aí o tempo é dominado pela comunicação e não pelo cálculo. No Spark, a mesma ideia não ajudou: a versão ficou 8% mais lenta com 1 núcleo e 11% com 32. Num teste local, a alteração reduziu o número de jobs por iteração de 8 para cerca de 5, mas os jobs eliminados eram leves, e numa comparação cuidadosa (com aquecimento e alternando a ordem das versões) a diferença ficou dentro da variação entre medidas. A mesma área deu resultados opostos porque, no MPI, o trabalho repetido era conta, e no Spark o custo está na coordenação, não nas contas.

6.2 Alteração 2: divisão dos dados

No MPI, esta foi a alteração mais importante. Com 8 processos, a versão ficou 2,1 vezes mais rápida que a original, e com 128 processos 5,7 vezes. Cada processo cuida de uma faixa de páginas de destino, então a troca por iteração passa a ser um `Allgatherv` em que cada processo envia só a sua faixa, e o vetor em que cada processo escreve tem só o tamanho dessa faixa, o que aproveita melhor a memória cache. As faixas são cortadas pelo número de links, e não de páginas: no nosso grafo, faixas com o mesmo número de páginas deixariam o primeiro de 4 processos com 63% dos links. O limite desse balanceamento é a página mais popular, que recebe cerca de 1% dos links e não pode ser dividida. Dentro de cada faixa, os links ficam em ordem de origem; numa primeira versão sem esse cuidado, a leitura das notas pulava pela memória e a alteração chegou a ser mais lenta que a original. O custo é a distribuição inicial, que passou de 0,16 s para 1,50 s por causa da ordenação dos links (fora do tempo de cálculo).

No Spark, a alteração 2 deixou a versão 27% mais rápida com 1 núcleo, mas com 32 núcleos empatou com a original (22,7 s contra 23,2 s). Pelo plano de execução, ela elimina a reorganização dos 10 milhões de links entre partições a cada iteração; com 32 núcleos, porém, todas as versões chegam ao mesmo piso de coordenação. O efeito também depende do tamanho dos dados: em testes preliminares, num grafo de 100 mil páginas o Spark já enviava a tabela de notas inteira sozinho (ela cabia no limite automático de 10 MB) e a alteração quase não fez diferença.

6.3 Alteração 3: formato dos dados

No MPI, serializar o vetor antes de enviar deixou a versão 8% mais lenta com 1 processo, 47% com 4 e 2,2 vezes mais lenta com 128. O efeito é menor do que o relatado em [1] porque lá foram serializadas listas de objetos Python, enquanto aqui se serializa um vetor NumPy, que já é um bloco contínuo de bytes. No Spark, a versão com RDD levou 102,6 s com 32 núcleos, 4,4 vezes o tempo da versão com DataFrames: cada link vira um objeto Python e cada operação é feita pelo Python, um registro por vez.

6.4 Versão otimizada

No MPI, a versão otimizada foi a mais rápida de todas: 0,326 s com 8 processos, 2,1 vezes mais rápida que o melhor tempo da original e 5,7 vezes mais rápida que a versão sequencial. No Spark, a otimizada

foi 26% mais rápida com 1 núcleo e 5% com 32, praticamente o efeito da alteração 2.

7. Limitações

Todas as medições usaram um único nó, porque o n-10 estava fora do ar. Por isso não houve comunicação pela rede: o MPI trocou dados pela memória do próprio nó e o Spark rodou em modo local, sem mestre e trabalhadores separados. Com 16 núcleos físicos, os resultados acima de 16 processos misturam o efeito do paralelismo com o do *hyper-threading* e, acima de 32, com o de processos dividindo núcleos. O grafo é sintético e de um único tamanho; grafos reais, com outra distribuição de links, podem se comportar de outro jeito. Por limite de tempo, usamos 10 iterações, 2 repetições no Spark e medimos as alterações do Spark só com 1 e 32 núcleos (a versão RDD só com 32, uma vez). Os números de notebook citados no texto são de testes preliminares e não entram nas tabelas.

8. Conclusões

O MPI foi muito mais rápido que o Spark neste problema: 68 vezes, comparando as melhores versões. Em troca, deu mais trabalho: a divisão do grafo, o balanceamento de carga e a comunicação foram escritos à mão, enquanto no Spark o mesmo algoritmo é uma sequência de operações de tabela e a divisão dos dados é feita pelo próprio Spark. O Spark teve speedup relativo maior, mas isso veio de um ponto de partida muito mais lento, dominado pela coordenação de cada iteração; escalar bem não significou ser rápido.

As alterações mostraram que o efeito de uma otimização depende de onde está o gargalo. Evitar contas repetidas ajudou o MPI, que gastava tempo em conta, e não ajudou o Spark, que gastava tempo em coordenação. Dividir os dados por destino foi a mudança mais importante nas duas abordagens, porque atacou a comunicação no MPI e a troca de dados no Spark, e o seu efeito no Spark dependeu do tamanho do grafo. Serializar dados custou pouco com vetores NumPy no MPI e muito com objetos Python no Spark.

Referências

[1] M. Saxena, S. Jha, S. Khan, J. Rodgers, P. Lindner e E. Gabriel. Comparison of MPI and Spark for Data Science Applications. Em *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, p. 682-690, 2020.

[2] S. Brin e L. Page. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1-7):107-117, 1998.

[3] L. Dalcin, R. Paz e M. Storti. MPI for Python. *Journal of Parallel and Distributed Computing*, 65(9):1108-1115, 2005.

[4] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker e I. Stoica. Spark: Cluster computing with working sets. Em *HotCloud*, 2010.

Apêndice: como reproduzir

O README do repositório explica cada passo. No nó do cluster, a preparação do ambiente e as medições usam os scripts abaixo; ao terminar, o segundo script gera a tabela-resumo (resultados/resumo_cluster.csv) e os gráficos usados neste relatório.

```
bash cluster/preparar.sh
nohup bash cluster/medir_tudo.sh > medicao.log 2>&1 &
```